

PAYFLOW

Güncel Proje Gelişim, Mimari Kararlar ve Yol Haritası Raporu

Üretim odaklı Java backend mühendisliği yaklaşımıyla geliştirilen
simüle dijital cüzdan ve ödeme platformu

Alan	Güncel bilgi
Rapor sürümü	4.0 - Transactional outbox, Kafka, observability ve alert delivery güncellemesi
Rapor tarihi	20 Temmuz 2026
Proje sahibi	Nursena Duyku
Repository	github.com/nursena-pc/payflow
Kararlı release	v0.3.0 - Transactional Outbox Milestone
Develop HEAD	414e29e - Alertmanager notification delivery (#38)
Aktif branch	docs/project-report-v4
Güncel durum	Outbox/Kafka milestone release edildi; monitoring ve alert delivery develop üzerinde tamamlandı.

Önemli kapsam notu

PayFlow gerçek para işlemez. Eğitim ve portföy amacıyla tasarlanmış bir finansal sistem simülasyonudur. Bu rapor, 17 Temmuz 2026 tarihli v3 raporunu v0.3.0 release, tamamlanan transactional outbox/Kafka zinciri, Prometheus-Grafana görünürlüğü ve Alertmanager-Mailpit bildirim kanıtlarıyla senkronize eder. Released main ile post-release develop çıktıları açıkça ayrıştırılmıştır.

İçindekiler

1. Yönetici Özeti ve Güncel Durum
2. Ürün Vizyonu, Sınırlar ve Başarı Ölçütleri
3. Mimari ve Teknoloji Seçimleri
4. Tamamlanan Çalışmalar ve Release Durumu
5. Kimlik, Güvenlik ve Wallet Yönetimi
6. Transfer ve Double-Entry Ledger
7. Transaction History ve API Sözleşmesi
8. Atomiklik, Idempotency ve Concurrency Garantileri
9. Test, OpenAPI ve Postman Güvence Modeli
10. GitHub ve Profesyonel Geliştirme Süreci
11. Mimari Kararlar ve ADR Durumu
12. Transactional Outbox Milestone
13. Publisher Lifecycle, Kafka ve Scheduling
14. Observability, Dashboard ve Prometheus Alert Rules
15. Alertmanager, Inhibition ve Mailpit Delivery
16. Bundan Sonraki Yol Haritası
17. Riskler, Teknik Borçlar ve Öncelikler
18. Definition of Done ve Backlog
19. Sonuç ve Doğrulama Notları

1. Yönetici Özeti ve Güncel Durum

PayFlow; kimlik ve wallet örneğinin ötesine geçerek atomik transfer, double-entry ledger, idempotency, sayfalı işlem geçmişi, OpenAPI sözleşmesi, durable transactional outbox, Kafka event publishing ve üretim benzeri operasyonel görünürlük sunan bütünlüklü bir backend platformuna dönüşmüştür.

v0.3.0 ile transactional outbox milestoneu main üzerinde etiketlenmiştir. Sonraki develop dilimlerinde outbox metrikleri, Grafana dashboardu, Prometheus alert kuralları, Alertmanager severity routing, inhibition ve Mailpit tabanlı yerel e-posta teslimatı tamamlanmıştır.

Başlık	Durum	Kanıt
Kimlik ve güvenlik	Tamamlandı	Register, login, RSA JWT, /users/me, deny-by-default
Wallet yönetimi	Release edildi	Open, summary, top-up, optimistic locking
Transfer + ledger	Release edildi	Atomik transfer, replay, rollback, concurrency
Transaction history	Release edildi	Pagination, direction/status/date filters
Outbox + Kafka	v0.3.0	Persistence, atomic integration, lifecycle, publisher
Monitoring	develop	Metrics, Grafana dashboard, 5 Prometheus alerts
Alert delivery	develop	Alertmanager routing + Mailpit + smoke test

2. Ürün Vizyonu, Sınırlar ve Başarı Ölçütleri

2.1 Ürün vizyonu

Nihai hedef; güvenli kimlik doğrulama, cüzdan yönetimi, simüle para yükleme, kullanıcılar arası transfer, değişmez muhasebe kayıtları, güvenilir event publishing, idempotent consumer davranışı ve üretim benzeri operasyonel araçlar içeren profesyonel bir backend platformu ortaya çıkarmaktır.

2.2 Bilinçli sınırlar

- Gerçek banka, kart, ödeme kuruluşu veya para transfer ağı entegrasyonu yapılmaz.
- Gerçek para tutulmaz; finansal lisans veya mevzuat uyumluluğu iddiası yoktur.
- İlk aşamada mikroservis yerine modüler monolit kullanılır.
- Kafka ve Redis yalnız açık bir güvenilirlik veya performans problemi çözdüklerinde devreye alınır.
- Wallet balance güncel durum projectionıdır; kalıcı finansal açıklanabilirlik ledger kayıtlarıyla sağlanır.
- Mailpit yalnız yerel geliştirme ve teslimat doğrulaması içindir; production e-posta sistemi değildir.

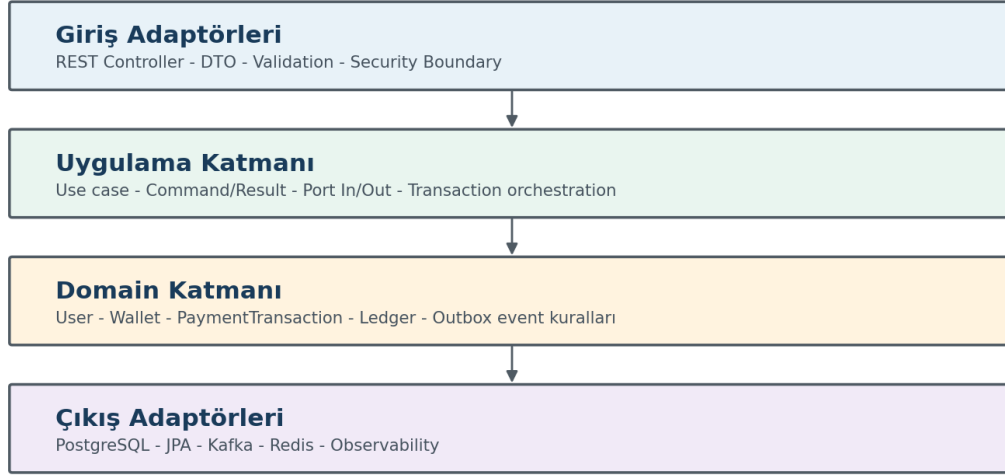
2.3 Başarı ölçütleri

- Aynı transfer isteği ikinci kez finansal hareket üretmez.
- Eşzamanlı güncellemelerde lost update oluşmaz.
- Her tamamlanan transfer için dengeli bir DEBIT/CREDIT çifti vardır.
- Ledger veya outbox persistence başarısızsa transfer bütünüyle rollback olur.
- Kafka geçici olarak erişilemez olsa bile durable event niyeti PostgreSQL içinde korunur.
- Publisher at-least-once davranır; consumerlar eventId ile idempotent çalışmak zorundadır.
- Outbox backlog, polling failure, stale event ve terminal failure operasyonel olarak görünürdür.
- Kritik garantiler gerçek PostgreSQL ve Kafka üzerinde deterministik testlerle kanıtlanır.

Neden bu alan seçildi?

Dijital cüzdan problemi; domain modelleme, transaction sınırları, concurrency, database constraint, güvenlik, event-driven tasarım ve operasyonel görünürlük gibi backend mühendisliğinin temel konularını tek projede görünür kılar.

3. Mimari ve Teknoloji Seçimleri



3.1 Modüler monolit ve hexagonal yön

User, wallet, transaction, ledger ve outbox modülleri aynı deployable uygulama içinde bulunur; ancak package-by-feature ve port/adapter sınırlarıyla ayrılır. Finansal transferin tek PostgreSQL transactionı içinde güvenli yürütülmesi korunurken Kafka, scheduling ve observability ayrıntıları domain katmanına sızmaz.

3.2 Güncel modül haritası

Modül	Bugünkü sorumluluk	Durum
common	ApiError ve ortak business exception altyapısı	Aktif
configuration	Security, Clock, OpenAPI ve runtime configuration	Aktif
user	Kayıt, login, profil, BCrypt, RSA JWT	Tamamlandı
wallet	Open, summary, top-up, Money, optimistic locking	Tamamlandı
transaction	Transfer, idempotency, history, orchestration	Tamamlandı
ledger	Immutable DEBIT/CREDIT kayıtları	Tamamlandı
outbox	Persistence, claim/lease, retry, publisher, metrics	Tamamlandı
notification / audit	Kafka consumerları ve eventId deduplication	Planlandı

3.3 Teknoloji seçimlerinin gerekçesi

Teknoloji	Neden kullanılıyor?
Java 21	Güncel LTS, güçlü tip sistemi, record ve modern dil özellikleri.
Spring Boot 3.5.x	Web, security, JPA, validation, transaction, scheduling ve test ekosistemini bütünleştirmek.
PostgreSQL 17	Transaction, constraint, index, JSONB, TIMESTAMPTZ ve concurrency doğruluğu.
Flyway	Şemayı kodla birlikte sürümlü ve tekrarlanabilir yönetmek.
Spring Security + OAuth2 RS	Stateless Bearer JWT doğrulamasını standart filtre zinciriyle uygulamak.
RSA JWT + BCrypt 12	Asimetrik token doğrulaması ve adaptif parola hashleme.
Testcontainers	Gerçek PostgreSQL ve Kafka davranışını CI ve yerelde tekrar üretmek.
Docker Compose	Uygulama, veri servisleri ve monitoring stackini standart yerel ortamda çalıştırmak.
Spring Kafka	Outbox kayıtlarını wallet.transfer.completed topicine yayınlamak.
Micrometer + Prometheus	Outbox publisher durumunu ölçmek ve alert rule üretmek.
Grafana	8 panelli provisioned transactional outbox dashboardu sunmak.
Alertmanager + Mailpit	Severity routing, inhibition ve yerel SMTP teslimatını doğrulamak.
OpenAPI + Postman	API sözleşmesini keşfedilebilir ve dış istemci akışını tekrar çalıştırılabilir yapmak.

Mimari ilke

Teknoloji, yalnız çözmesi gereken açık problem bulunduğunda eklenir. Kafka transfer use case içinde doğrudan çağrılmaz; önce durable outbox niyeti PostgreSQL içine yazılır. Alerting katmanı da finansal doğruluğu değil, operasyonel görünürlüğü destekler.

4. Tamamlanan Çalışmalar ve Release Durumu

Alan	Tamamlanan çıktı
Repository ve CI	README, CONTRIBUTING, SECURITY, şablonlar, Maven Wrapper, GitHub Actions clean verify.
Yerel altyapı	PostgreSQL 17, Redis, Kafka, app, Prometheus, Grafana, Alertmanager ve Mailpit.
Kimlik / Wallet	RSA JWT, user profile, wallet open/summary/top-up, optimistic locking.
Transfer / Ledger	Idempotent atomik transfer, double-entry ledger, rollback ve concurrency garantileri.
History / API	Pagination, filtreler, Swagger UI, /v3/api-docs ve Postman workflow.
Transactional outbox	Persistence, transfer entegrasyonu, lifecycle, orchestration, Kafka publisher, polling.
Observability	Micrometer metrics, backlog ölçümü, dashboard ve Prometheus alert rules.
Alert delivery	Alertmanager severity routing, inhibition, Mailpit ve tekrar kullanılabilir smoke test.

4.1 Release geçmişi ve güncel Git durumu

Sürüm / ref	Commit	Anlamı
main / v0.1.0	8fc299f	Wallet Management Milestone
main / v0.2.0	218f68b	Transfer, ledger ve transaction history milestone
main / v0.3.0	81f7edf	Transactional Outbox Milestone
develop	414e29e	Alertmanager notification delivery (#38)
aktif branch	docs/project-report-v4	Raporu güncel durumla senkronize etme

Release disiplini

Feature PRLarı squash merge ile developa alınır. Milestone releaseinde develop -> main merge commit kullanılır ve main üzerinde semantic version tag oluşturulur. v0.3.0 sonrası develop, monitoring ve alerting çalışmalarını taşıyan v0.4.0 geliştirme hattıdır.

5. Kimlik, Güvenlik ve Wallet Yönetimi

5.1 Güvenli varsayılanlar

- Uygulama stateless çalışır; form login ve HTTP Basic kapalıdır.
- Public endpointler açıkça listelenir; yeni yollar varsayılan olarak kapalıdır.
- Actuator erişiminde yalnız health ve Prometheus scrape endpointleri açıkça izinlidir.
- Parolalar BCrypt cost 12 ile hashlenir.
- JWT RSA signature, issuer ve expiration ile doğrulanır.
- Owner kimliği JWT subjectinden türetilir; request bodyden kabul edilmez.
- DTO/response whitelist yaklaşımıyla yalnız güvenli alanlar dışarı çıkar.

5.2 Wallet davranışı

Endpoint	Yetki	Davranış
POST /api/v1/wallets	Bearer JWT	Sıfır bakiyeli wallet açar; ikinci deneme 409.
GET /api/v1/wallets/me	Bearer JWT	Token sahibinin wallet özetini döndürür; yoksa 404.
POST /api/v1/wallets/me/top-ups	Bearer JWT	Pozitif amount ile simüle top-up; conflict 409.

5.3 Optimistic locking

wallets.version alanı JPA @Version ile yönetilir. Gerçek PostgreSQL concurrency testinde iki transaction aynı versionı okur; yalnız biri commit olur, diğeri WALLET_CONCURRENT_UPDATE ile reddedilir. Böylece lost update engellenir.

```
POST /api/v1/wallets/me/top-ups
Authorization: Bearer <access-token>
Content-Type: application/json

{ "amount": 250.00 }
```

6. Transfer ve Double-Entry Ledger

6.1 Transaction domain modeli

PaymentTransaction, TRANSFER türü ve PENDING/COMPLETED/FAILED statüleriyle modellenmiştir. IdempotencyKey boş değerleri ve 100 karakter üzerini reddeder. Transfer use case, kaynak wallet authenticated owner üzerinden bulur ve hedef wallet ile tutar kurallarını domain modelleri aracılığıyla doğrular.

6.2 Double-entry invariantı

Her tamamlanan transfer aynı transactionId altında iki değişmez kayıt üretir: kaynak wallet için DEBIT, hedef wallet için CREDIT. Tutar, currency ve transaction kimliği eşit; wallet kimlikleri farklıdır.

Bileşen	Uygulanan karar
payment_transactions	Source/target UUID, type, status, amount, currency, idempotency key, timestamps.
Idempotency uniqueness	UNIQUE (source_wallet_id, idempotency_key).
Distinct wallets	CHECK: source ve target aynı olamaz.
ledger_entries	Immutable JPA entity; update/delete portu yok.
Entry type	Yalnız DEBIT veya CREDIT.
Duplicate ledger guard	UNIQUE (transaction_id, wallet_id, entry_type).
Outbox bağlamı	Completed transfer için aynı transactionda tek durable event kaydı.

Finansal açıklanabilirlik

Wallet balance hızlı okuma için current-state projectiondır. Ledger paranın neden ve nasıl hareket ettiğini açıklayan kalıcı kayıt katmanıdır. Outbox ise bu tamamlanmış finansal gerçeğin dış sistemlere güvenilir biçimde duyurulmasını sağlar.

7. Transaction History ve API Sözleşmesi

7.1 Transaction history

GET /api/v1/transactions/me endpointi authenticated kullanıcının walletını JWT subject üzerinden çözer. Client başka bir wallet kimliği sağlayamaz. Sonuçlar createdAt DESC, id DESC sırasıyla deterministik döner.

Filtre	Davranış
page / size	Sayfalama ve sınır doğrulaması.
direction	INCOMING veya OUTGOING.
status	PENDING, COMPLETED veya FAILED.
from / to	UTC tarih aralığı; geçersiz aralık stable error.
response	Idempotency key içermez; güvenli public transaction alanları.

7.2 Güncel API yüzeyi

Metot	Endpoint	Yetki
GET	/api/v1/system/health	Public
POST	/api/v1/auth/register	Public
POST	/api/v1/auth/login	Public
GET	/api/v1/users/me	Bearer JWT
POST	/api/v1/wallets	Bearer JWT
GET	/api/v1/wallets/me	Bearer JWT
POST	/api/v1/wallets/me/top-ups	Bearer JWT
POST	/api/v1/transfers	Bearer JWT + Idempotency-Key
GET	/api/v1/transactions/me	Bearer JWT
GET	/actuator/health	Public operational endpoint
GET	/actuator/prometheus	Public scrape endpoint

8. Atomiklik, Idempotency ve Concurrency Garantileri

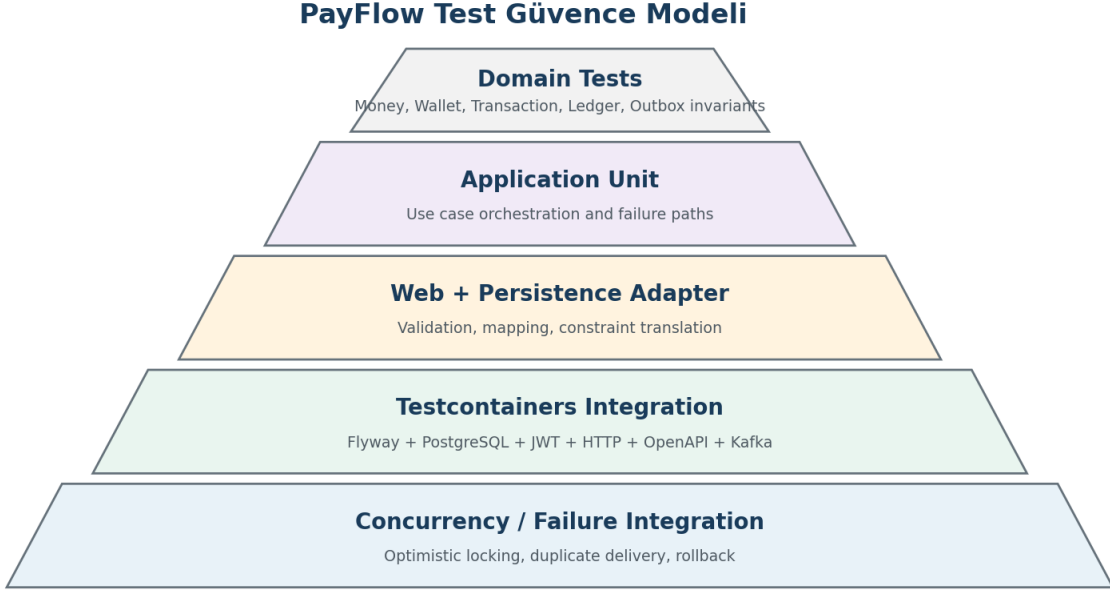
8.1 Atomik transfer sırası

1. Kaynak wallet authenticated ownerId ile bulunur.
2. Mevcut transaction source wallet + IdempotencyKey ile aranır.
3. Replay varsa payload eşleşmesi ve COMPLETED durumu doğrulanır.
4. Hedef wallet ve currency/active/self-transfer/balance kuralları doğrulanır.
5. PENDING payment transaction kaydedilir.
6. Kaynak debit ve hedef credit deterministik wallet ID sırasıyla persistence edilir.
7. Double-entry ledger kaydedilir.
8. Transaction COMPLETED yapılır.
9. wallet.transfer.completed event niyeti aynı PostgreSQL transactionında outboxa eklenir.

8.2 Doğrulanmış senaryolar

Senaryo	Beklenen davranış	Sonuç
İlk transfer	Tek transaction, iki wallet update, iki ledger entry, tek outbox event	Doğrulandı
Aynı key + aynı payload	Yeni hareket yok; aynı transaction replay	Doğrulandı
Aynı key + farklı payload	IDEMPOTENCY_KEY_CONFLICT	Doğrulandı
Ledger persistence hatası	Financial state ve outbox birlikte rollback	Doğrulandı
Outbox persistence hatası	Financial state ve ledger birlikte rollback	Doğrulandı
Eşzamanlı duplicate istek	Tek finansal hareket ve tek event niyeti	Doğrulandı
Eşzamanlı wallet update	Bir başarı, bir optimistic conflict	Doğrulandı

9. Test, OpenAPI ve Postman Güvence Modeli



9.1 Neden gerçek PostgreSQL ve Kafka?

Constraint adları, JSONB, TIMESTAMPTZ, UUID davranışı, transaction abort semantiği, optimistic locking, FOR UPDATE SKIP LOCKED ve Kafka ack/failure davranışı gömülü test veritabanıyla güvenilir biçimde kanıtlanamaz. Testcontainers bu projede kolaylık değil, doğruluk kararıdır.

9.2 Güncel doğrulama kanıtları

Kontrol	Sonuç
Maven kalite kapısı	mvnw clean verify - BUILD SUCCESS, 1:45
OpenAPI JSON	HTTP 200, application/json ve contract integration testi
Postman runner	15 request, 30 test, 0 failure
Prometheus config	1 rule file, 5 alert rule - promtool SUCCESS
Alertmanager config	1 inhibition rule, 3 receiver - amtool SUCCESS
Monitoring runtime	PayFlow target up; 1 active Alertmanager, 0 dropped
Notification smoke test	warning/critical x firing/resolved - 4 teslimat başarılı

10. GitHub ve Profesyonel Geliştirme Süreci

10.1 Uygulanan akış

1. develop güncellenir ve odaklı feat/docs/chore branch açılır.
2. Feature küçük checkpointlere ayrılır; her checkpoint statik ve runtime doğrulamayla kapanır.
3. Conventional Commit mesajları kullanılır ve branch remotea pushlanır.
4. Feature uçtan uca tamamlanmadan PR açılmaz.
5. PRda CI, conflict, kapsam, dokümantasyon ve test komutları doğrulanır.
6. Feature PRı squash merge ile developa alınır.
7. Milestone tamamlandığında develop -> main release PRı merge commit ile birleştirilir.
8. main üzerinde semantic version tag ve GitHub Release oluşturulur.

10.2 Outbox ve operasyonel görünürlük PR zinciri

PR	Kapsam	Sonuç
#28	Transactional outbox persistence	Merged
#29	Completed transfer -> outbox atomic integration	Merged
#30	Outbox event lifecycle	Merged
#31	Publisher orchestration	Merged
#32	Kafka publishing	Merged
#33	Configurable polling	Merged
#34	Outbox observability	Merged
#35	develop -> main v0.3.0 release	Merged + tag
#36	Start v0.4.0 development	Merged
#37	Transactional outbox monitoring	Merged
#38	Alertmanager notification delivery	Merged

Neden küçük PR?

Review kapsamı küçülür, hata kaynağı daha kolay bulunur, geri alma maliyeti düşer ve Git geçmişi projenin mimari gelişimini okunabilir biçimde taşır.

11. Mimari Kararlar ve ADR Durumu

ADR	Karar	Durum
ADR 0001	Modüler monolit ve hexagonal dependency yönü	Accepted / implemented
ADR 0002	Transferleri double-entry ledger ile kaydetme	Accepted / implemented
ADR 0003	Transfer idempotency ownershipunu PostgreSQL ile koruma	Accepted / implemented
ADR 0004	Transfer eventlerini transactional outbox üzerinden yayınlama	Accepted / implemented

11.1 ADR 0004 ana kararları

- Transfer use case doğrudan KafkaTemplate kullanmaz.
- Outbox kaydı finansal transferle aynı PostgreSQL transactionında yazılır.
- Kafka publication ayrı publisher tarafından commit sonrasında yapılır.
- Topic wallet.transfer.completed, event schema version 1 olarak sabitlenmiştir.
- At-least-once delivery kabul edilir; exactly-once iddiasında bulunulmaz.
- Consumerlar eventId ile idempotent çalışmak zorundadır.
- Publisher kayıtları PostgreSQL claim/lease mekanizması ve SKIP LOCKED yaklaşımıyla paylaşır.
- Retry, bounded delay ve terminal failure davranışı uygulanır.
- Amount event payloadında decimal string olarak taşınır.
- HTTP idempotency key event payloadına taşınmaz.

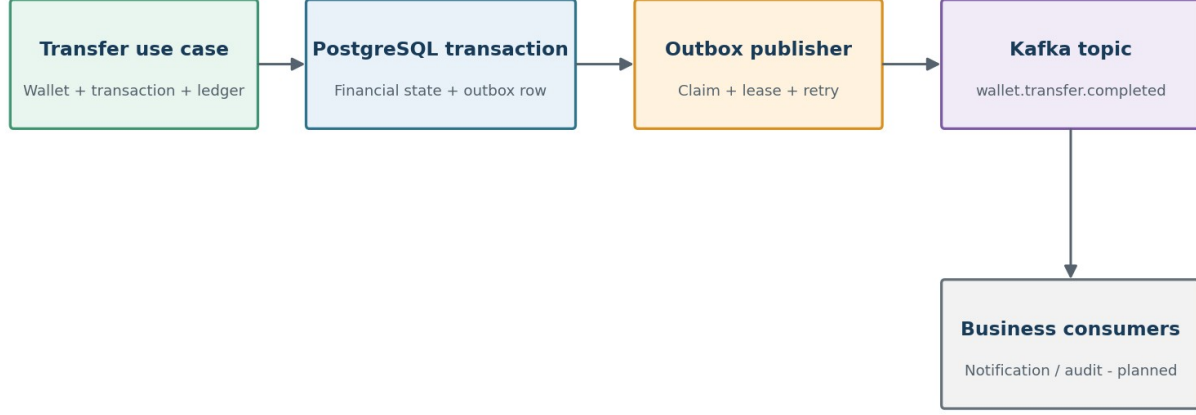
ADR neden önemlidir?

Kod yalnız ne yapıldığını gösterir. ADR, neden bu tasarımın seçildiğini, hangi alternatiflerin reddedildiğini ve hangi garanti/trade-offların kabul edildiğini gelecekteki geliştiricilere açıklar.

12. Transactional Outbox Milestone

Transactional Outbox - Güvenilir Event Akışı

Kafka erişilebilir olmasa bile event niyeti PostgreSQL içinde kaybolmadan saklanır.



12.1 Tamamlanan kapsam

Bileşen	Tamamlanan çıktı
PostgreSQL persistence	Flyway migration, JSONB payload, constraints, JPA adapter ve repository tests
Atomik entegrasyon	Completed transfer ile tek outbox event aynı transactionda
Event identity	eventId, eventType, version, aggregate/transaction kimliği
Delivery lifecycle	Claim, lease, retry scheduling, publication ve terminal failure
Publisher orchestration	Batch claim, per-event publish sonucu ve state transition
Kafka adapter	wallet.transfer.completed topicine event version 1 yayınlama
Runtime polling	Configurable batch size, lease, delay ve publisher identity
Observability	Polling, backlog, oldest age, retry ve terminal failure metrikleri

Temel garanti

Kafka geçici olarak kapalı olsa bile transferin event niyeti kaybolmaz. Finansal state ve outbox kaydı tek PostgreSQL transactionında commit olur; ağ yayını daha sonra güvenli biçimde tekrar denir.

13. Publisher Lifecycle, Kafka ve Scheduling

13.1 Claim, lease ve parallel publisher güvenliği

Publisher, yayınlanabilir outbox kayıtlarını batch halinde claim eder. PostgreSQL row locking ve SKIP LOCKED yaklaşımı aynı eventin iki publisher tarafından aynı anda işlenmesini önler. Lease süresi tamamlanan veya yarım kalan claimlerin daha sonra geri kazanılmasını sağlar.

Aşama	Davranış
Claim	Sınırlı batch; uygun status/availableAt; lock owner ve lock expiry metadata.
Publish success	Kafka ack sonrasında published metadata ve terminal başarılı durum.
Transient failure	Attempt artışı, hata bilgisi ve gelecekteki availableAt ile retry.
Lease recovery	Süresi dolan claim tekrar yayınlanabilir hale gelir.
Terminal failure	Retry sınırı sonrasında operasyonel müdahale gerektiren kalıcı durum.

13.2 Kafka event sözleşmesi

Alan	Karar
Topic	wallet.transfer.completed
Version	1
Delivery	At-least-once
Deduplication	Consumer tarafında eventId
Amount	Decimal string
Kimlikler	Event ve transaction kimlikleri; kaynak/hedef context
Hariç tutulan alan	HTTP Idempotency-Key

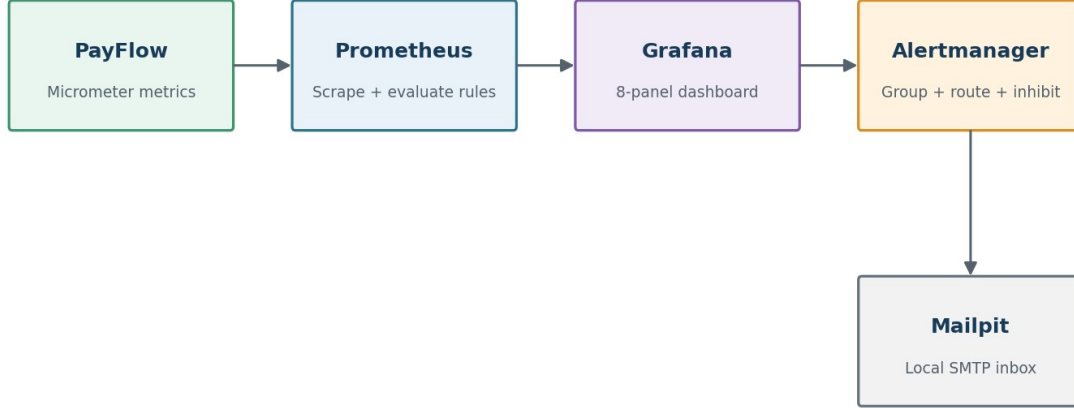
13.3 Runtime configuration

- Polling enable/disable flag ile yerel veya test ortamında scheduler kontrol edilir.
- Batch size, lease duration, fixed delay ve initial delay environment üzerinden ayarlanabilir.
- Publisher identity birden fazla instance ve log/metric korelasyonu için görünürdür.
- Retry davranışı kod içinde sınırlı ve deterministic olacak şekilde tasarlanmıştır.

14. Observability, Dashboard ve Prometheus Alert Rules

Operasyonel Görünürlük ve Yerel Bildirim Akışı

Outbox backlog, polling ve terminal failure sinyalleri dashboard ve alarmlara taşınır.



14.1 Ölçülen operasyonel sinyaller

Sinyal	Amaç
Polling execution	Scheduler çalışması ve başarısız polling sayısı
Publish outcome	Başarılı, retry ve terminal failure sonuçları
Backlog	Yayınlanmayı bekleyen event sayısı
Oldest event age	En eski yayınlanmamış eventin yaşı
Claim / lease	Publisher sahipliği ve stuck event işaretleri

14.2 Provisioned monitoring stack

Bileşen	Sürüm / kimlik	Rol
Prometheus	v3.13.1	PayFlow scrape, rule loading ve Alertmanager forwarding
Grafana	13.1.0-ubuntu	PayFlow klasörü ve 8 panelli provisioned dashboard
Dashboard UID	payflow-outbox-overview	Backlog, stale event, failures ve polling görünürlüğü

14.3 Alert rule seti

Alert	Severity	Anlamı
PayFlowMetricsTargetDown	critical	PayFlow metrics target scrape edilemiyor.
PayFlowOutboxPollingFailures	warning	Publisher polling hataları oluşuyor.
PayFlowOutboxBacklogHigh	warning	Yayınlanmamış event backlogu yükseldi.
PayFlowOutboxOldestEventStale	warning	En eski event kabul edilen yaşı aştı.
PayFlowOutboxTerminalFailures	critical	Terminal failure durumunda event bulunuyor.

15. Alertmanager, Inhibition ve Mailpit Delivery

Prometheus alert kurallarından üretilen aktif alarmlar Alertmanagera gönderilir. Alertmanager severity etiketine göre receiver seçer, aynı bağlamdaki alarmları gruplar, kritik target-down durumunda ikincil warning gürültüsünü inhibe eder ve yerel SMTP teslimatını Mailpit üzerinden kanıtlar.

15.1 Routing ve grouping

Severity	Receiver	Recipient
critical	critical-email	oncall@payflow.local
warning	warning-email	engineering@payflow.local
diğer / eksik severity	null	E-posta üretilmez
Ayar		Değer
SMTP smarthost		mailpit:1025
From		alertmanager@payflow.local
group_by		alertname, service, component, severity
group_wait / interval		10s / 30s
repeat_interval		4h
send_resolved		true

15.2 Inhibition davranışı

PayFlowMetricsTargetDown critical olarak firing durumundayken aynı service ve component etiketlerine sahip warning alarmları inhibe edilir. Uygulama tamamen erişilemezken aynı kök nedenden kaynaklanan ikincil backlog veya polling uyarılarının e-posta gürültüsü üretmesi engellenir.

15.3 Teslimat smoke testi

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
-File scripts/observability/test-alertmanager-notifications.ps1
```

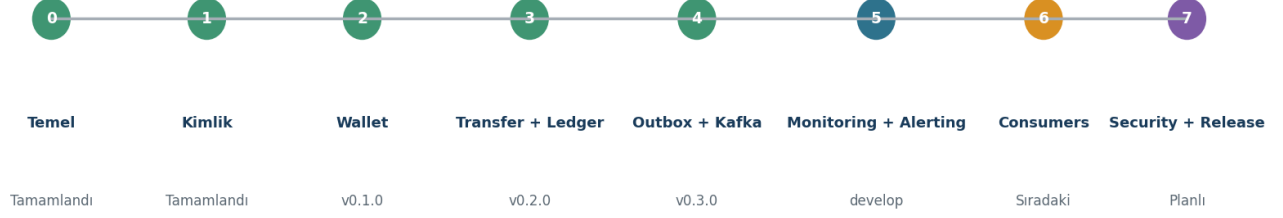
- warning firing -> engineering@payflow.local
- critical firing -> oncall@payflow.local
- warning resolved -> engineering@payflow.local
- critical resolved -> oncall@payflow.local

Yerel geliştirme kapsamı

Mailpit authentication ve TLS olmadan çalışan yerel test SMTP servisi. Production notification delivery için environment-specific credentials, TLS, secret management ve operasyonel sahiplik gerekir.

16. Bundan Sonraki Yol Haritası

PayFlow Teslimat Yol Haritası - 20 Temmuz 2026



16.1 Yakın vadeli sıra

1. Kafka notification ve audit consumerları; eventId tabanlı idempotency store.
2. Consumer failure/retry stratejisi ve duplicate delivery integration testleri.
3. Published outbox retention/archival politikasının ölçümle belirlenmesi.
4. Structured JSON logging ve request/event correlation ID.
5. Refresh token rotation, logout/revocation ve Redis tabanlı login rate limiting.
6. API security hardening ve performance/load test senaryoları.
7. v0.4.0 release branchi, Maven versioning standardı ve release metadata doğrulaması.

16.2 Sonraki ürün dilimleri

- Business event consumers ile kullanıcı bildirim ve audit projectionları.
- Operational alerting için production receiver stratejisi ve secret yönetimi.
- Demo deployment, image hardening ve runtime configuration matrisi.
- CHANGELOG ve release-notes otomasyonu.
- Distributed tracing yalnız ihtiyaç ve maliyet analizi sonrasında.

17. Riskler, Teknik Borçlar ve Öncelikler

Risk / borç	Etkisi	Önerilen aksiyon
Business consumer idempotency yok	Duplicate event işleme riski	eventId unique processing store ve integration test
Published outbox retention belirsiz	Tablo büyümesi	Metric ve hacim sonrası retention/archival policy
v0.3.0 Maven metadata geride	Release artifact kimliği tutarsız	v0.4.0 release branchinde version standardı
Refresh token yok	Access token yaşam döngüsü eksik	Rotation ve revocation storage stratejisi
RSA key local/in-memory	Production riski	Secret tabanlı key loading ve rotation
Mailpit yalnız local	Production alert teslimatı yok	TLS/credential/receiver ownership tasarımı
Tracing ve correlation sınırlı	Dağıtık hata analizi zorlaşabilir	Structured logs + correlation ID, sonra tracing
Bazı integration testleri uzun	CI süresi büyüyebilir	Test kategorileri, paralellik ve kontrollü reuse
Dokümantasyon drift riski	Portföy görünürlüğü	Her milestone sonunda report/README/roadmap sync

Öncelik ilkesi

Önce finansal doğruluk ve güvenlik, sonra reliable event delivery, ardından operasyonel görünürlük ve performans. Açık bir problem çözmeyen teknoloji projeye eklenmez.

18. Definition of Done ve Backlog

18.1 Her feature için Definition of Done

- Acceptance criteria ve hata senaryoları nettir.
- Domain/application/web/persistence sınırları korunur.
- Başarılı yol ve kritik failure path testleri vardır.
- Gerekliyse gerçek PostgreSQL/Redis/Kafka integration testi bulunur.
- Configuration dosyaları promtool/amtool/Compose gibi native araçlarla doğrulanır.
- mvnw clean verify yerelde başarılıdır.
- git diff --check temiz ve working tree kontrollüdür.
- Commitler odaklı ve Conventional Commit formatındadır.
- PR açıklaması kapsamı, riskleri ve test kanıtlarını içerir.
- CI başarılı, conflict yok ve review tamamdır.
- README/OpenAPI/Postman/ADR/monitoring/rapor etkileri senkronize edilmiştir.

18.2 Öncelikli backlog

Öncelik	Branch / issue	Kapsam
P0	feat/kafka-consumers	Notification/audit consumers ve eventId deduplication
P0	test/consumer-delivery	Duplicate, retry ve failure integration scenarios
P1	feat/refresh-token	Rotation, revocation ve storage
P1	feat/login-rate-limit	Redis IP/account limit ve Retry-After
P1	feat/structured-logging	JSON logs ve correlation ID
P2	chore/outbox-retention	Published event archival/cleanup policy
P2	chore/project-versioning	Maven version/release metadata standardı
P2	chore/release-pipeline	CHANGELOG, image hardening, demo deployment

19. Sonuç ve Doğrulama Notları

PayFlow v0.3.0 itibarıyla yalnız endpoint sayısını artıran bir CRUD projesi değildir. Aynı transfer isteğinin ikinci kez finansal hareket üretmemesi, ledger/outbox hatasında yarım state kalmaması, concurrent duplicate istekte tek finansal hareket oluşması ve event niyetinin Kafka kesintisinde kaybolmaması gerçek PostgreSQL ve Kafka odaklı testlerle güvence altına alınmıştır.

Post-release develop çalışmaları, reliable event delivery zincirini operasyonel açıdan görünür hale getirmiştir. Prometheus-Grafana ile backlog ve failure sinyalleri izlenmekte; Alertmanager-Mailpit zinciri warning ve critical alarmların hem firing hem resolved teslimatını otomatik smoke test ile kanıtlamaktadır.

19.1 Doğrulama kaynakları

- Git status, log, commit, push, PR ve release çıktıları.
- mvnw clean verify kalite kapısı - 20 Temmuz 2026 BUILD SUCCESS.
- Gerçek PostgreSQL ve Kafka Testcontainers integration testleri.
- OpenAPI JSON metadata ve contract integration testi.
- Postman Runner: 15 request, 30 passed, 0 failed.
- promtool: 1 rule file ve 5 alert rule SUCCESS.
- amtool: 1 inhibition rule ve 3 receiver SUCCESS.
- Prometheus: PayFlow target up; Alertmanager discovery 1 active / 0 dropped.
- Notification smoke test: warning/critical firing/resolved teslimatları başarılı.
- ADR 0001-0004, README, monitoring ve alerting dokümantasyonu.

Güncel repository özeti	Değer
Kararlı release	v0.3.0
main HEAD / tag	81f7edf
develop HEAD	414e29e - feat: add Alertmanager notification delivery (#38)
Aktif geliştirme	docs/project-report-v4
Rapor kapsam tarihi	20 Temmuz 2026

Rapor sürüm notu

Bu belge v0.3.0 transactional outbox releaseini ve develop üzerindeki monitoring/alert delivery tamamlanmış durumunu temsil eder. Business notification/audit consumers ve eventld deduplication bir sonraki ana geliştirme dilimidir.